OSSP (Operating Systems And Systems Programming) - 25CS2104E

SKILL

WEEK-7

1.To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

```
aruntej@LAPTOP-DGRT8C5G:~$ mkdir -p ~/Week7
cd ~/Week7
aruntej@LAPTOP-DGRT8C5G:~/Week7$ nano task1.c
aruntej@LAPTOP-DGRT8C5G:~/Week7$ gcc -Wall -Wextra task1.c -o task1
./task1
WEEK-7 Task 1: Directory and Exit Handler
Current Directory: /home/aruntej/Week7
Enter command (pwd, save, exit): pwd
Current Path: /home/aruntej/Week7
Enter command (pwd, save, exit): hello
Invalid command. Try pwd, save, or exit.
Enter command (pwd, save, exit): pwd
Current Path: /home/aruntej/Week7
Enter command (pwd, save, exit): save
Directory state saved successfully.
Enter command (pwd, save, exit): hello
Invalid command. Try pwd, save, or exit.
Enter command (pwd, save, exit): exit
Exit requested. Cleaning up resources...
Program terminated successfully.
```

C code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

#define PATH_SIZE 4096

int main(void) {
   char cwd[PATH_SIZE];
   char command[100];

   printf("WEEK-7 Task 1: Directory and Exit Handler\n");

   // Retrieve and display current directory
   if (getcwd(cwd, sizeof(cwd)) == NULL) {
```

```
    perror("getcwd failed");
    return EXIT_FAILURE;
}

printf("Current Directory: %s\n", cwd);

while (1) {
    printf("Enter command (pwd, save, exit): ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL) {
        printf("\nInput ended. Exiting safely.\n");
        break;
    }

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "pwd") == 0) {
        if (getcwd(cwd, sizeof(cwd)) != NULL)
            printf("Current Path: %s\n", cwd);
        else
            perror("getcwd failed");
    }
    else if (strcmp(command, "save") == 0) {
        FILE *file = fopen("directory_state.txt", "w");

        if (file == NULL) {
            perror("Unable to save state");
            continue;
        }

        if (getcwd(cwd, sizeof(cwd)) == NULL) {
            perror("getcwd failed");
            fclose(file);
            continue;
        }

        fprintf(file, "Current Directory: %s\n", cwd);

        if (fclose(file) != 0)
            perror("Error closing state file");
        else
            printf("Directory state saved successfully.\n");
    }
    else if (strcmp(command, "exit") == 0) {
        printf("Exit requested. Cleaning up resources...\n");
        break;
    }
```

```
        else {
            printf("Invalid command. Try pwd, save, or exit.\n");
        }
    }

    printf("Program terminated successfully.\n");
    return EXIT_SUCCESS;
}
```

2.To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
aruntej@LAPTOP-DGRT8C5G:~/Week7$ nano task2.c
aruntej@LAPTOP-DGRT8C5G:~/Week7$ gcc -Wall -Wextra task2.c -o task2
./task2
WEEK-7 Task 2: Environment Export Handler
Commands: export NAME=value, show NAME, child NAME, exit
export> export NAME=ArunTej
show NAME
chVariable already exists; updating it.
NAME=ArunTej exported successfully.
export> NAME=ArunTej
export> ild NAME
export NAME=Student
show NAME
child NAME
export 123NAME=wrong
exitArunTej
Child process completed successfully.
export> Variable already exists; updating it.
NAME=Student exported successfully.
export> NAME=Student
export> Student
Child process completed successfully.
export> Invalid variable name.
export> exit
Unknown command.
export> ^C
```

C code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/types.h>
```

```
#include <sys/wait.h>

#define MAX 1024

// Validate environment variable names
int valid_name(const char *name) {
    if (name == NULL || name[0] == '\0')
        return 0;

    if (!(isalpha((unsigned char)name[0]) || name[0] == '_'))
        return 0;

    for (int i = 1; name[i] != '\0'; i++) {
        if (!(isalnum((unsigned char)name[i]) ||
              name[i] == '_'))
            return 0;
    }

    return 1;
}

int main(void) {
    char input[MAX];

    printf("WEEK-7 Task 2: Environment Export Handler\n");
    printf("Commands: export NAME=value, show NAME, ");
    printf("child NAME, exit\n");

    while (1) {
        printf("export> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        // Parse export syntax
        if (strncmp(input, "export ", 7) == 0) {
            char *assignment = input + 7;
            char *equal = strchr(assignment, '=');

            if (equal == NULL) {
                printf("Invalid syntax. Use export NAME=value\n");
                continue;
```

```
    }

    *equal = '\0';
    char *name = assignment;
    char *value = equal + 1;

    if (!valid_name(name)) {
        printf("Invalid variable name.\n");
        continue;
    }

    // Detect whether the variable already exists
    if (getenv(name) != NULL)
        printf("Variable already exists; updating it.\n");

    if (setenv(name, value, 1) == 0)
        printf("%s=%s exported successfully.\n",
               name, getenv(name));
    else
        perror("setenv failed");
}
else if (strncmp(input, "show ", 5) == 0) {
    char *name = input + 5;
    char *value;

    if (!valid_name(name)) {
        printf("Invalid variable name.\n");
        continue;
    }

    value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("Variable not found.\n");
}
else if (strncmp(input, "child ", 6) == 0) {
    char *name = input + 6;

    if (!valid_name(name)) {
        printf("Invalid variable name.\n");
        continue;
    }

    pid_t pid = fork();

    if (pid < 0) {
```

```
            perror("fork failed");
            continue;
        }

        if (pid == 0) {
            // Child reads the inherited environment variable
            char *args[] = {"printenv", name, NULL};
            execvp(args[0], args);

            perror("execvp failed");
            _exit(127);
        } else {
            int status;

            if (waitpid(pid, &status, 0) == -1) {
                perror("waitpid failed");
                continue;
            }

            if (WIFEXITED(status) &&
                WEXITSTATUS(status) == 0) {
                printf("Child process completed successfully.\n");
            } else {
                printf("Variable unavailable to child, "
                       "or execution failed.\n");
            }
        }
    }
    else {
        printf("Unknown command.\n");
    }
}

printf("Environment handler terminated.\n");
return 0;
}
```